

Introduction to Python (for Data Analysis)

Alireza Hosseinzadeh

alireza.hosseinzadeh@manchester.ac.uk

12 June 2026



Q-Step

MANCHESTER
1824

The University of Manchester

Let's Get to Know Each Other

Please briefly introduce yourself:

- Your name
- Your course, programme, or research area
- One hobby or interest outside study/ work
- Your experience with Python or any other programming language
- What kind of data you work with, or expect to work with
- Why you joined this course
- One thing you would like to be able to do with Python by the end of today

Course Summary

This course provides foundation-level skills in computer programming using Python, with a particular focus on data analysis. Students will be introduced to core programming concepts, including variables, data types, functions, loops, errors, debugging, and the use of packages. The course also covers practical skills for working with data, including importing datasets, summarising data, cleaning and manipulating data, and producing basic visualisations.

Alongside technical Python skills, the course introduces good programming practice, including writing clear and readable code, documenting work, testing code, and using tools such as Jupyter Notebook, Anaconda, and GitHub. By the end of the course, students should be able to use Python to carry out basic data analysis tasks and understand how programming can support reproducible and efficient research.

Course Objectives

By the end of this course, students should be able to:

- Understand what computer programming is and how Python is used for data analysis.
- Explain basic programming concepts, including variables, data types, functions, loops, packages, and errors.
- Use Python to import, explore, summarise, clean, and manipulate datasets.
- Produce basic data visualisations to support data exploration and communication.
- Read and interpret Python code with sufficient confidence to check whether it performs the intended task.
- Use tools such as Anaconda, Jupyter Notebook, and GitHub to support Python-based workflows.
- Apply good programming practice, including clear code structure, documentation, debugging, and basic testing.
- Understand how Python and Large Language Models (LLMs) can be used together to support code development, while recognising the need to review, test, and validate generated code.
- Appreciate the role of reproducible and sustainable coding practices in research and data analysis.

Learning Outcomes

- Foundation-level understanding of computer programming using Python.
- Understanding of key Python concepts, including variables, data types, functions, packages, loops, and errors.
- Practical ability to use Python for basic data analysis tasks.
- Awareness of how Python supports reproducible and efficient research workflows.

Skills Outcomes

- Import, explore, summarise, clean, and manipulate datasets using Python.
- Produce basic visualisations to explore data quality and communicate results.
- Read and interpret Python code to check whether it performs the intended task.
- Identify, interpret, and debug common Python errors.
- Use Jupyter Notebook, Anaconda, and Python packages in a structured workflow.
- Document code clearly and apply basic testing principles.
- Use large language models effectively to support code development, while critically reviewing and validating the generated code.

Course Content/Syllabus

- Introduction to computer programming and Python.
- Installing Python and working with Anaconda and Jupyter Notebook.
- Variables, data types, and basic scripting.
- Functions, modules, packages, and reusable code.
- Flow control, loops, and basic procedures.
- Errors, debugging, and tracing code.
- Reading and importing data into Python.
- Exploring, summarising, cleaning, and manipulating datasets.
- Data visualisation for exploratory analysis and communication.
- Good coding practice: documentation, testing, and reproducibility.
- Using GitHub and version control for managing code.
- Using Large Language Models (LLMs) to support code development and review.

Main Themes

- What is Python and how do I use it?
- How do I turn a research question into clear coding steps?
- How do I read, clean, explore, and visualise data?
- How do I write code that is readable, reproducible, and easier to debug?
- How can Large Language Models (LLMs) support Python programming without replacing critical code review?

Learning Outcomes: First Half

By the end of the first half of the course, students should be able to:

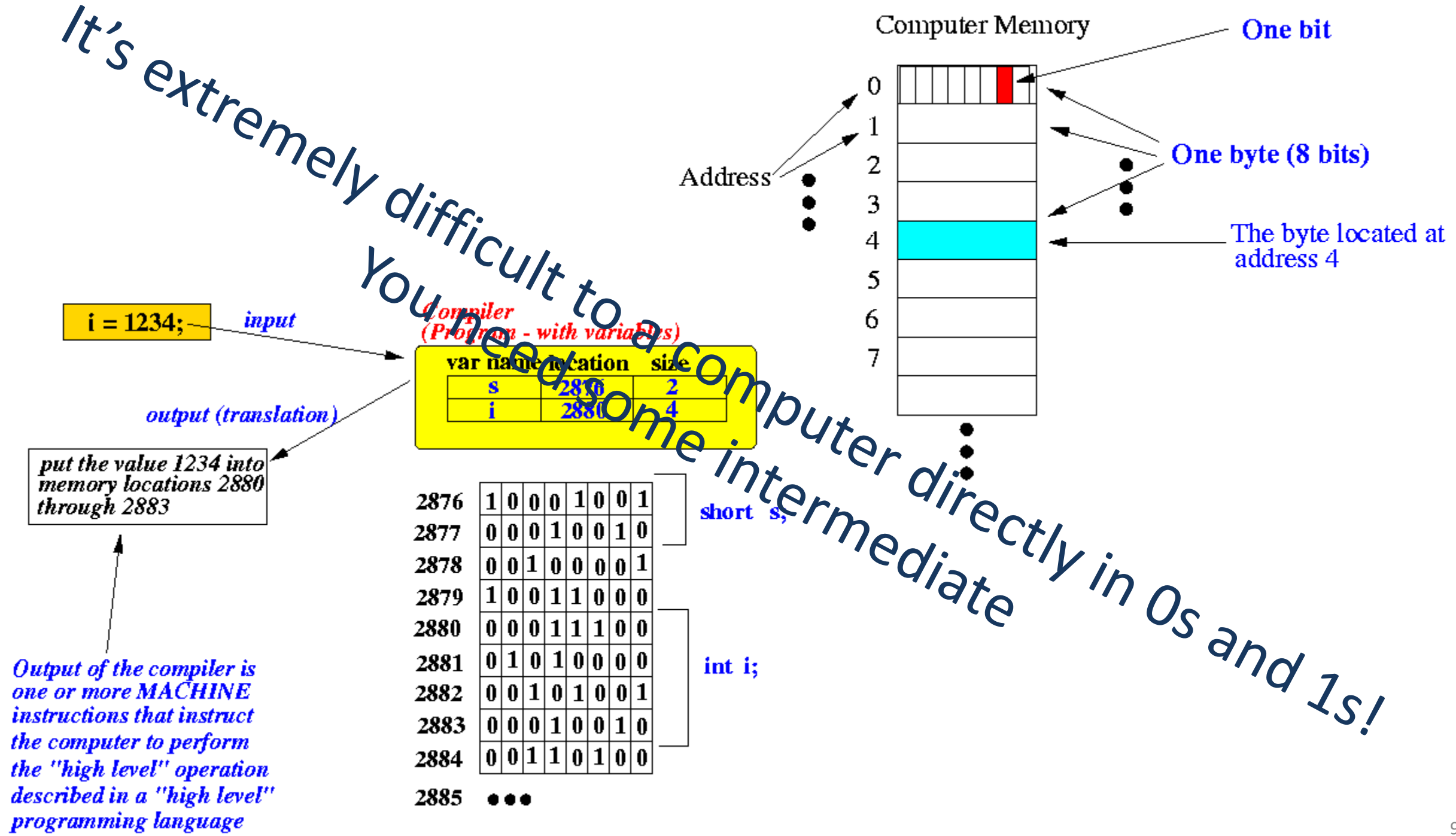
- Explain what programming is and why Python is useful for data analysis.
- Understand the difference between writing code, running code, and interpreting output.
- Recognise Python as a high-level, interpreted, dynamically typed programming language.
- Work with basic Python objects, including variables, strings, numbers, lists, tuples, dictionaries, and Boolean values.
- Understand how functions, modules, and packages support reusable code.
- Use Anaconda and Jupyter Notebook to write and run Python code.
- Identify common Python errors and use error messages to begin debugging code.
- Describe coding tasks clearly enough to use large language models effectively as coding assistants.
- Critically review generated code rather than treating it as automatically correct.

Starting from the Beginning... What is Programming?

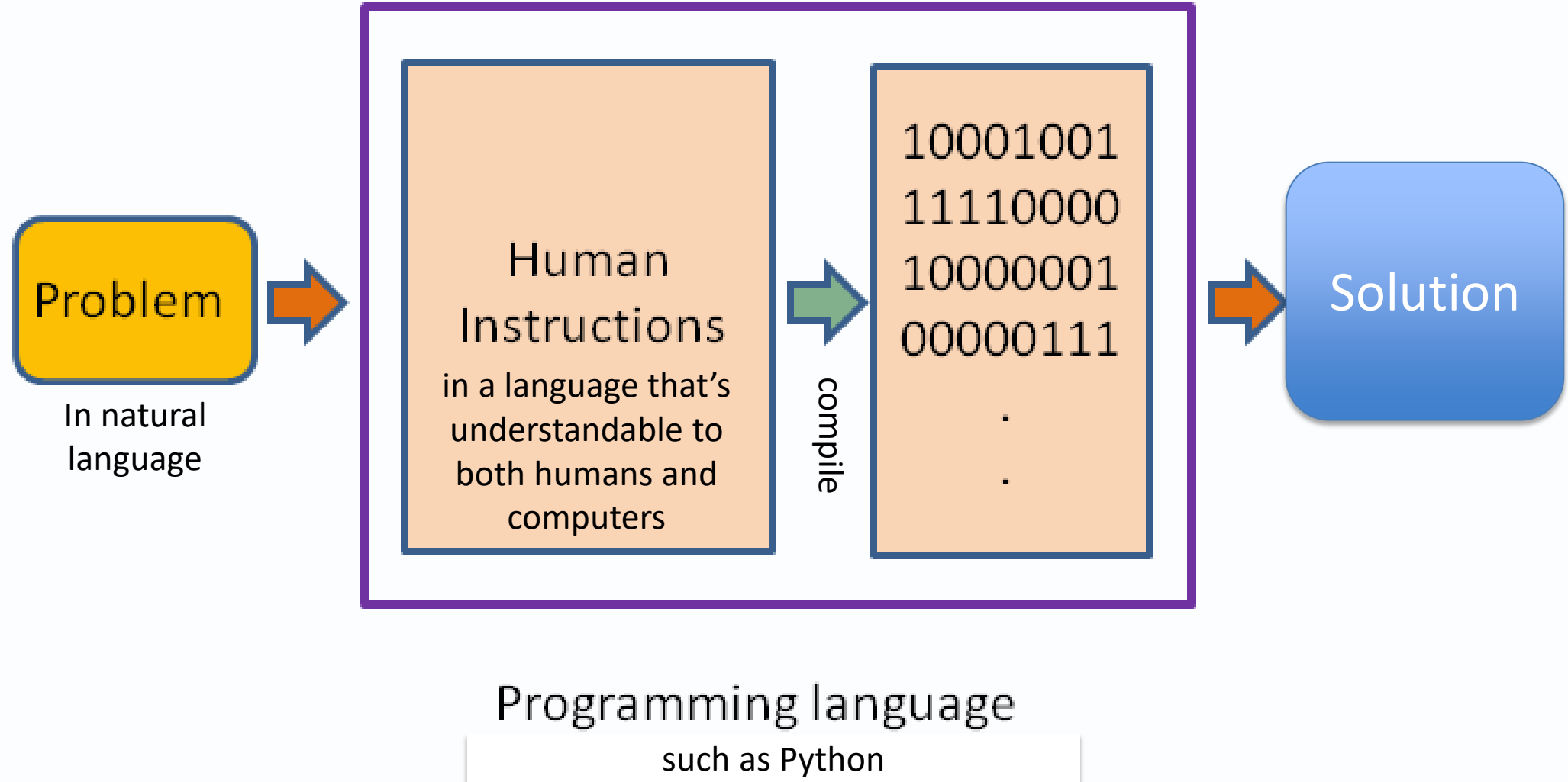


It's extremely difficult to a computer directly in 0s and 1s!

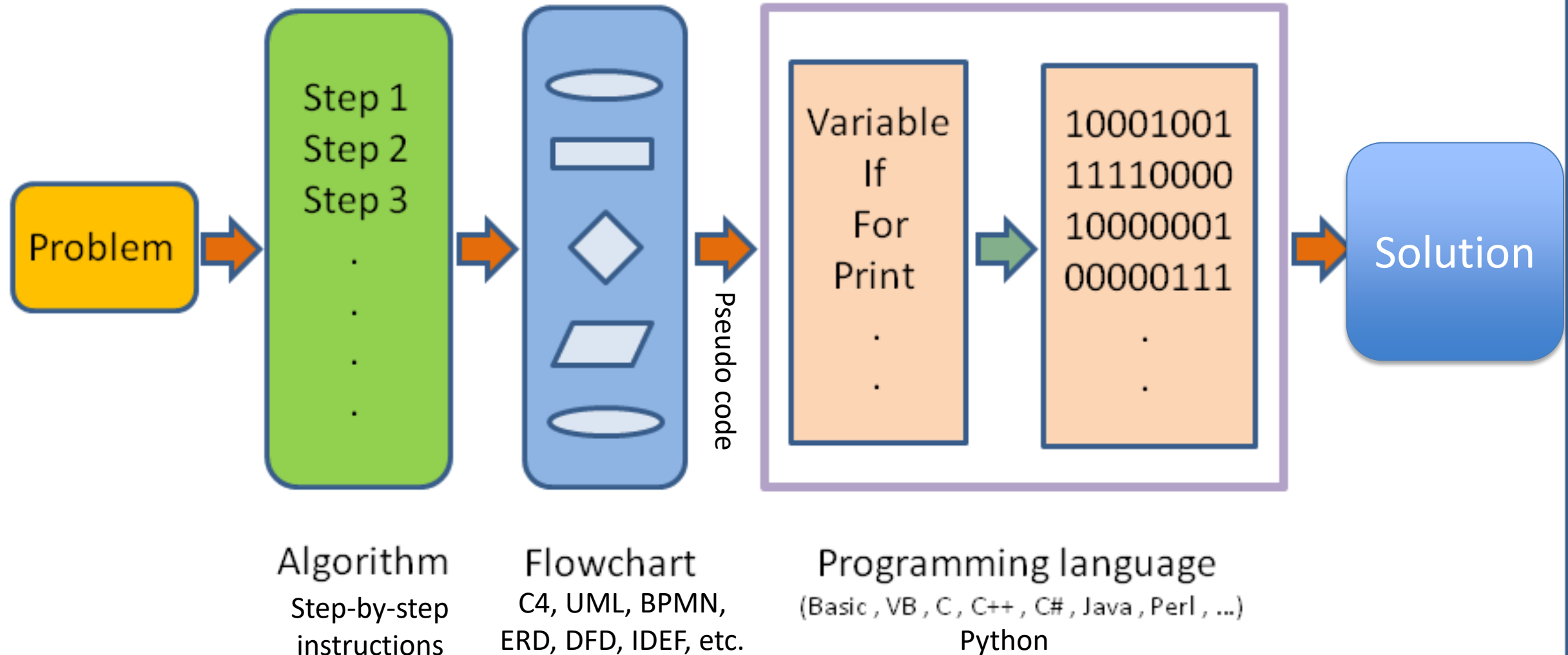
You need some intermediate



Asking a computer to do things for us

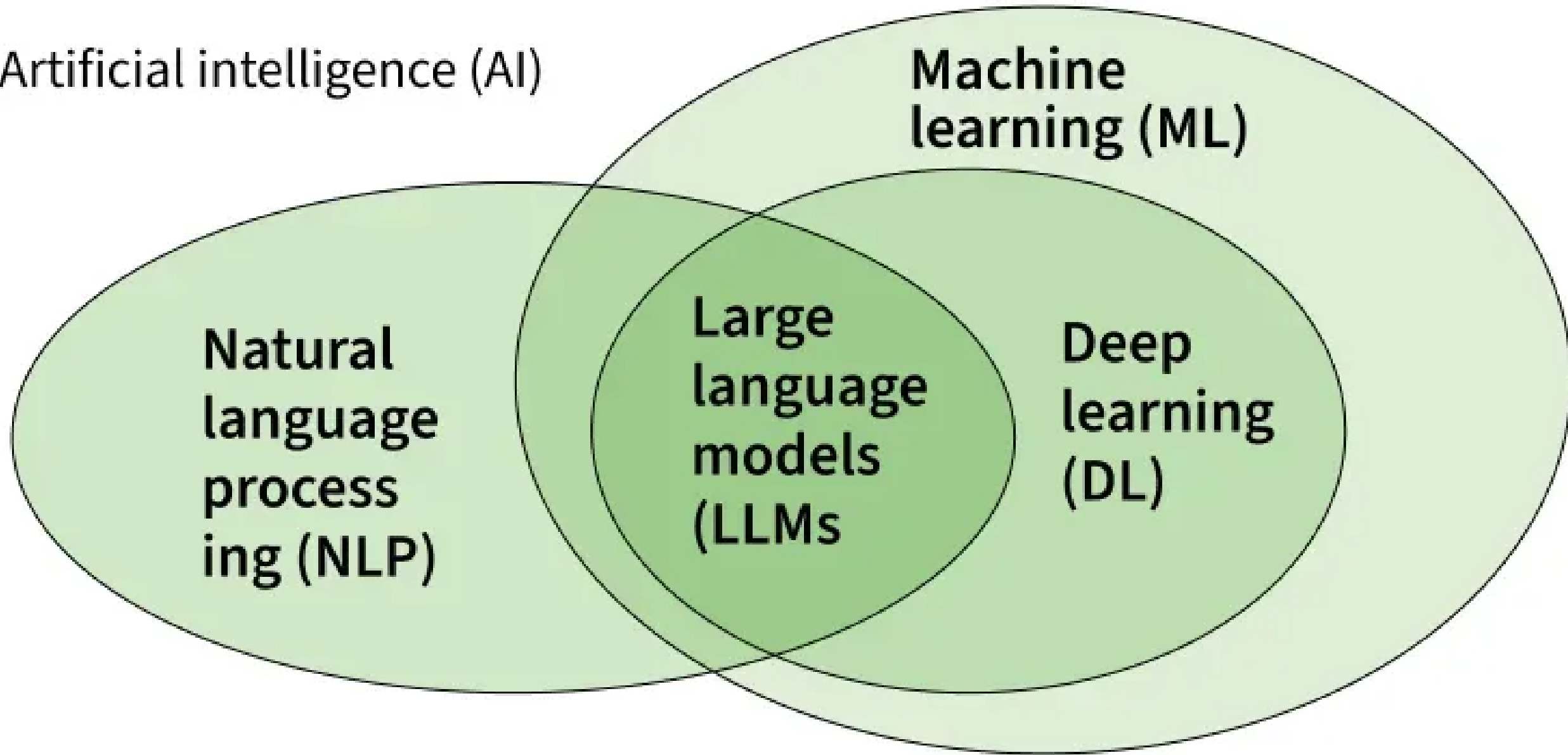


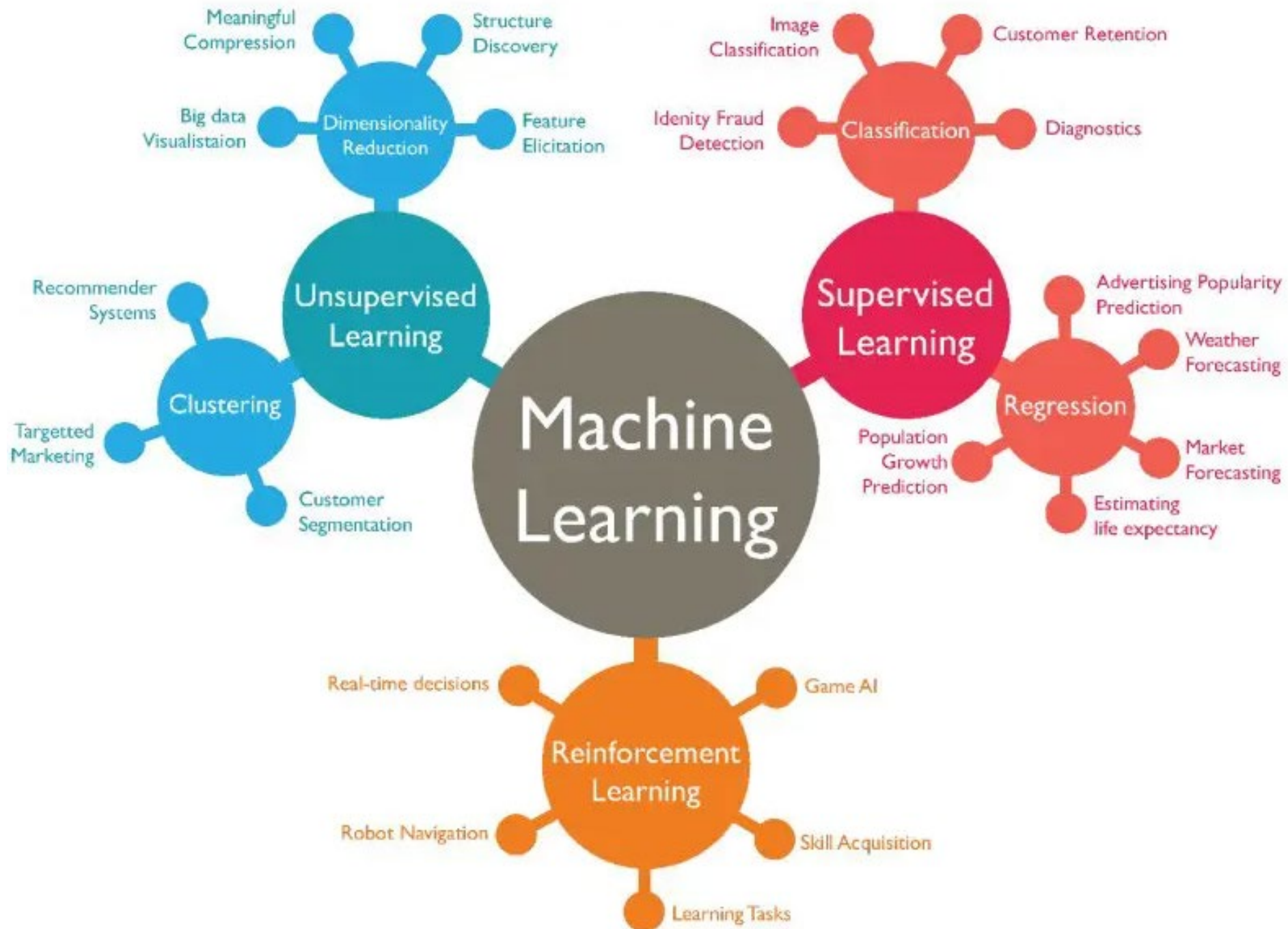
Converting your natural language request to specific steps



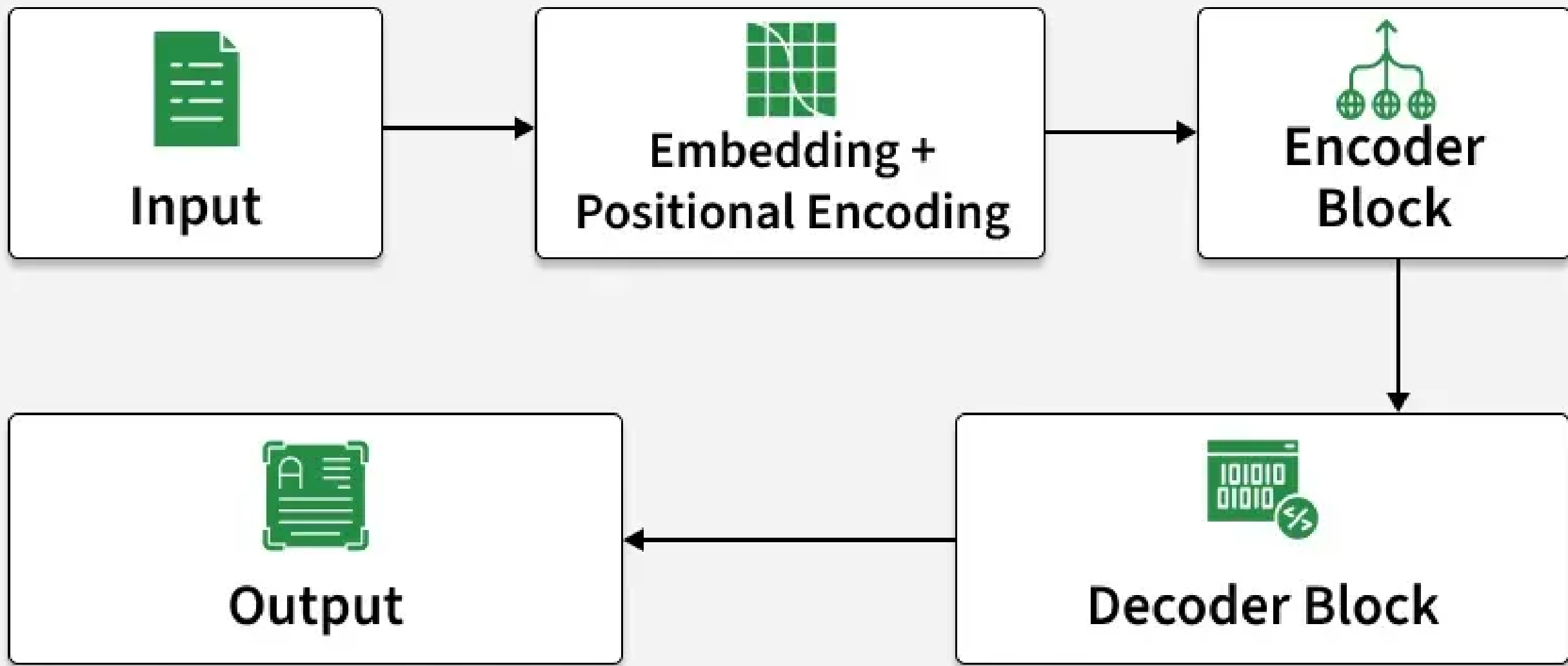


Artificial intelligence (AI)





Transformers in LLMs



Some programming terminology



Source code is the human-readable form of a computer program

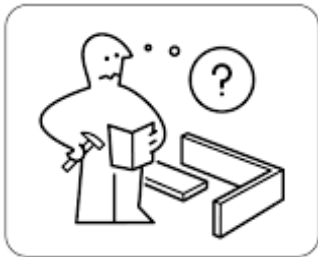
A computer program is a sequence or set of instructions in a programming language for a computer to execute

A screenshot of a Notepad window titled "hello_world - Notepad". The window has a menu bar with "File", "Edit", "Format", "View", and "Help". The text area contains two lines of Python code:

```
print ( "Hello World" )  
print ( "Welcome to Python" )|
```

Programming is the writing of computer programs

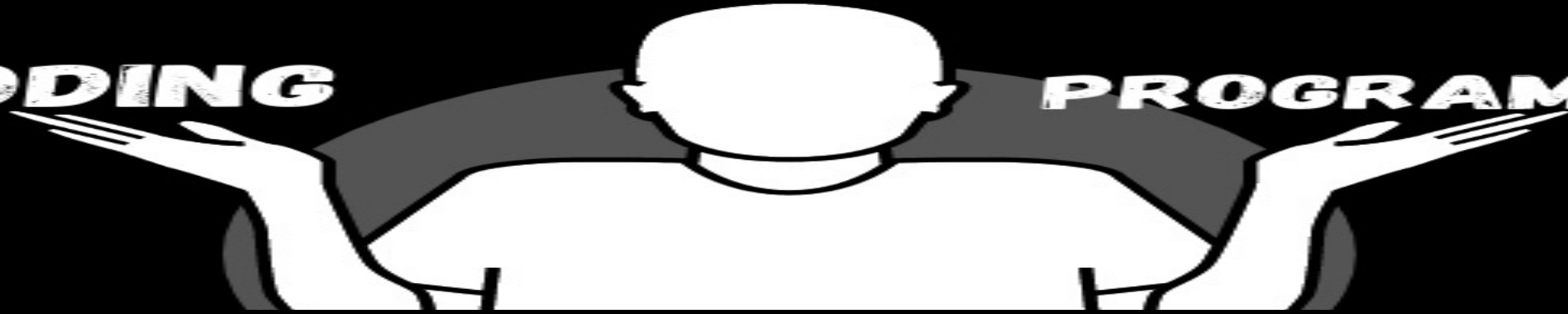
That makes you a programmer!



DIFFERENCES

CODING

PROGRAMMING



Narrow (Subset of programming)

Scope

Broad (The entire lifecycle)

Code editors / IDEs

Primary Tool

Algorithms, design patterns, testing frameworks, and compilers

Write functional syntax

**Main
Objective**

Solve complex problems and build systems

A script, a UI component, or a simple function

Outcome

A fully executable software application

DATA



SORTED



ARRANGED



PRESENTED
VISUALLY



EXPLAINED
WITH A STORY



THE DATA SCIENCE PROCESS

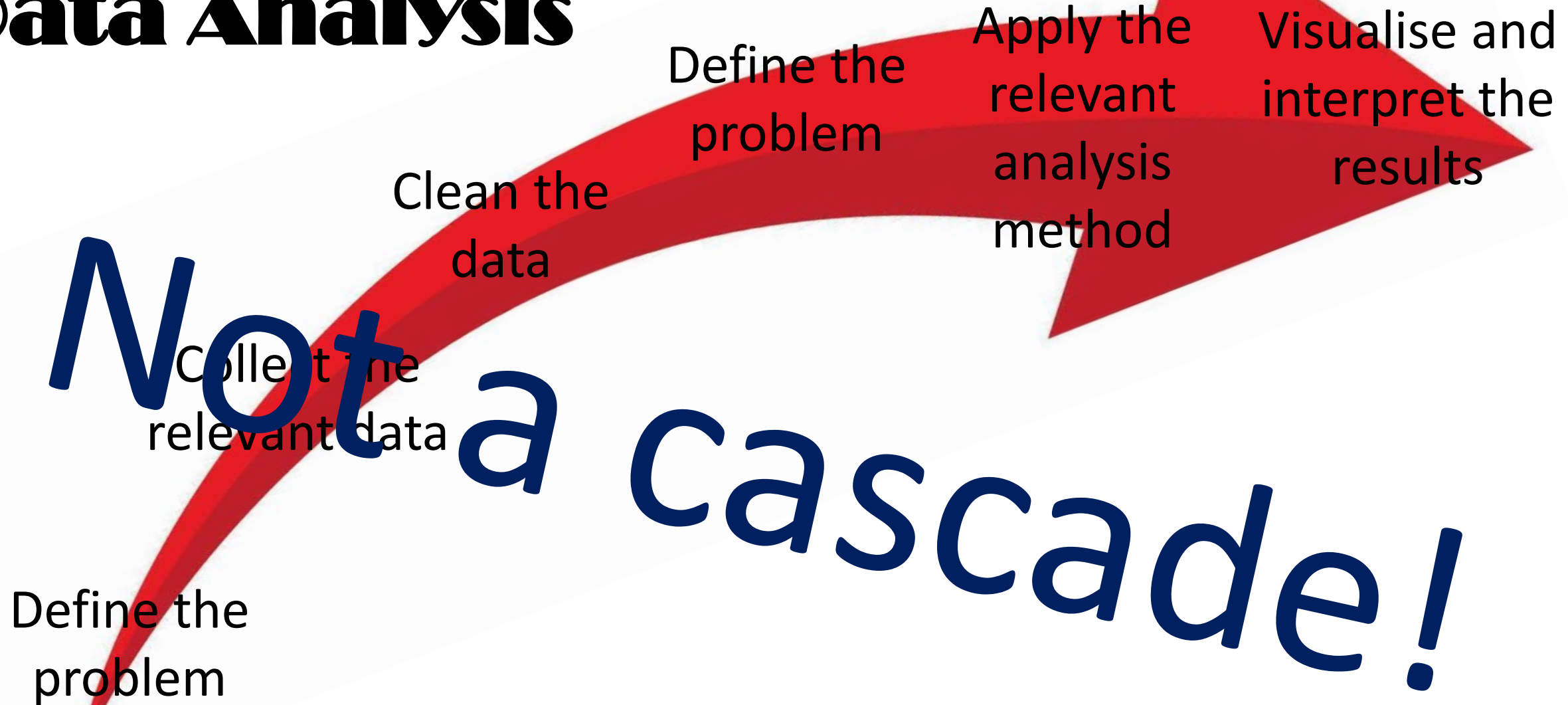


Data Engineers

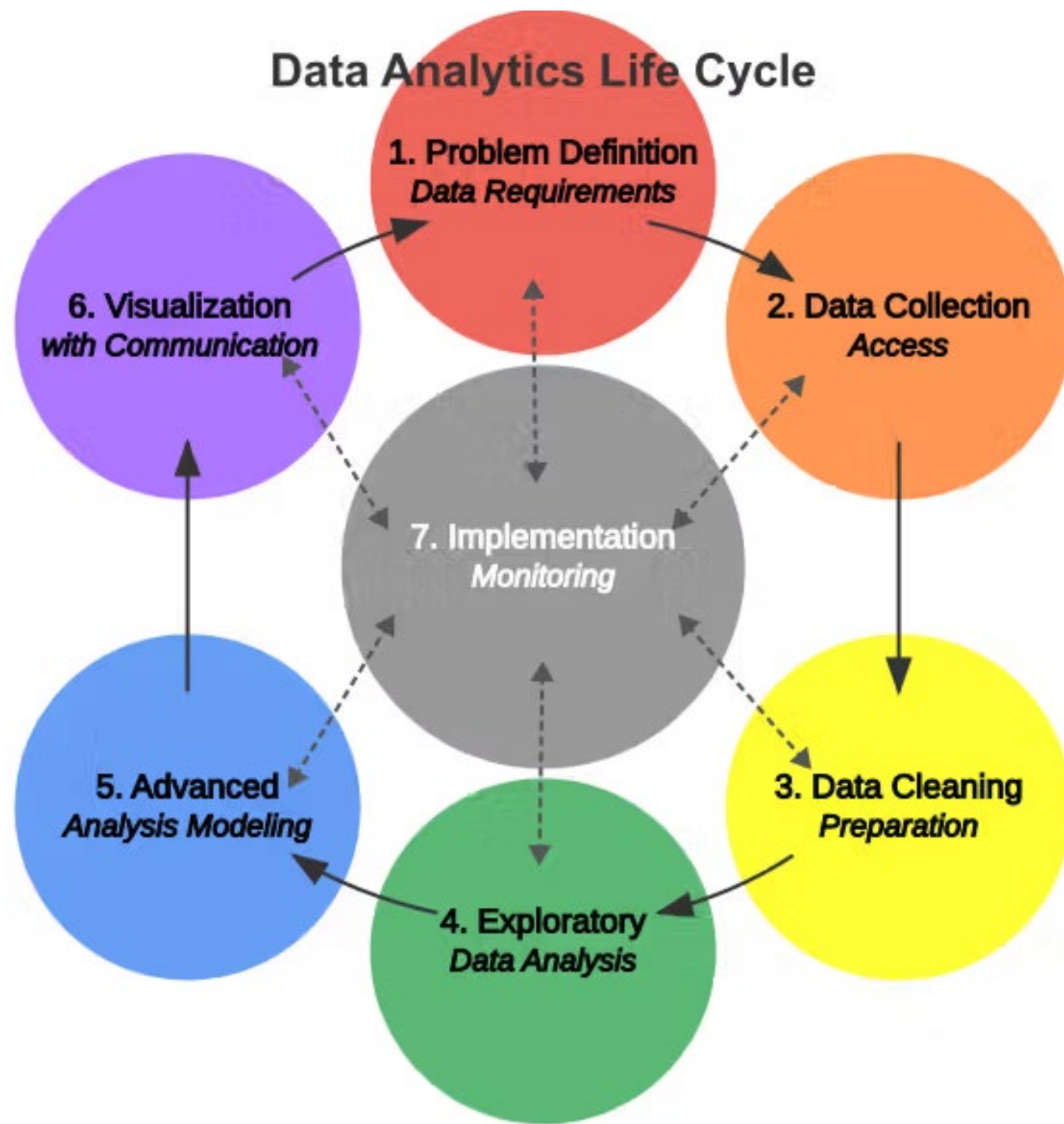
Data Analysts

Machine Learning Engineers

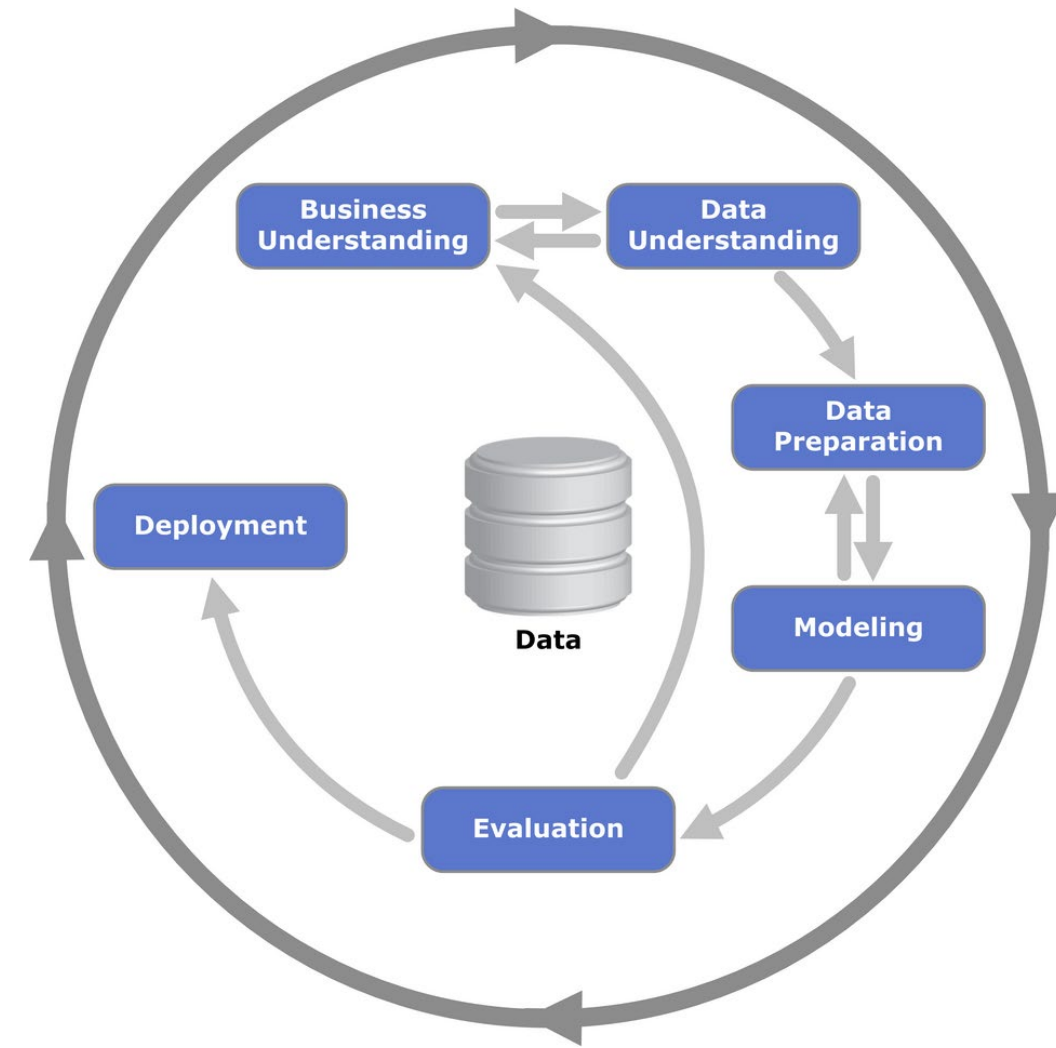
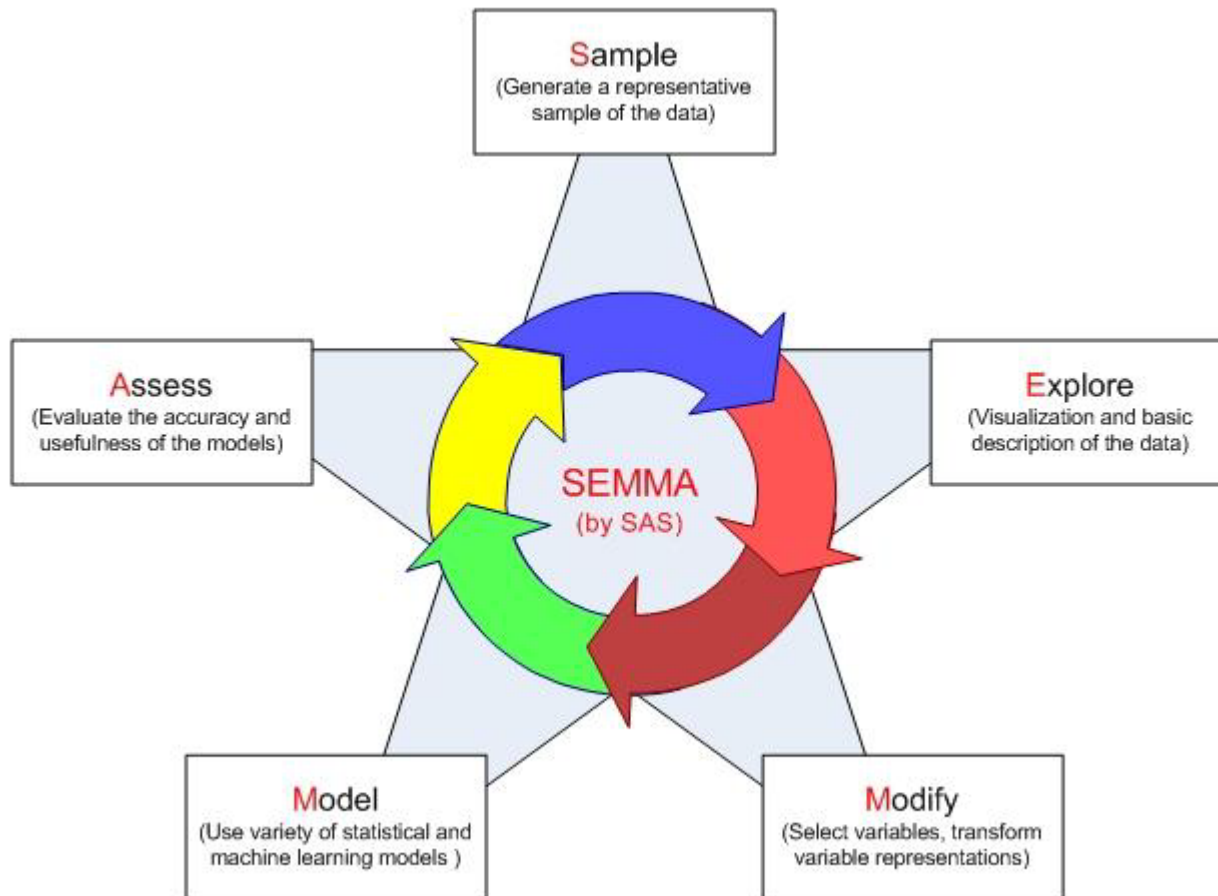
The Steps in Data Analysis



Data Analytics Life Cycle



SEMMA (Sample, Explore, Modify, Model, and Assess)



CRISP-DM (Cross-Industry Standard Process for Data Mining)

- A programming language

- ✓ A programme is a sequence of instructions that specify how to perform a computation

- Which is....

- ❑ First released in 1991 by Guido Van Rossum named after his favourite BBC show “Monty Python's Flying Circus” to make it mysterious and cool!
- ❑ Initially began as a hobby project but now a powerful programming language which powers Instagram, Google & YouTube, Spotify, Netflix, Dropbox, Reddit, Global Finance, ...
- ❑ A general-purpose programming language mainly developed for data processing/ analysis.
- ❑ Guido described Python as a programming language that was designed for people who do not have a background in computer science or software engineering.
- ❑ Differs from other programming languages:
 - ❖ Dynamically typed
 - ❖ Interpreted
 - ❖ High-level

What is Python?

High-level:

Similar to natural (human) language and not machine code (0s and 1s) Easier to read and understand.

Dynamically typed:

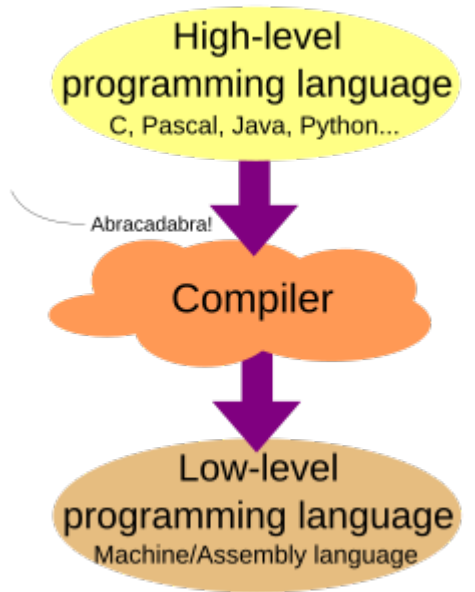
In Python you don't need to declare a variable's type Code is shorter and more concise but also more prone to errors (explained further when talking about primitive data types and in practice)

Interpreted:

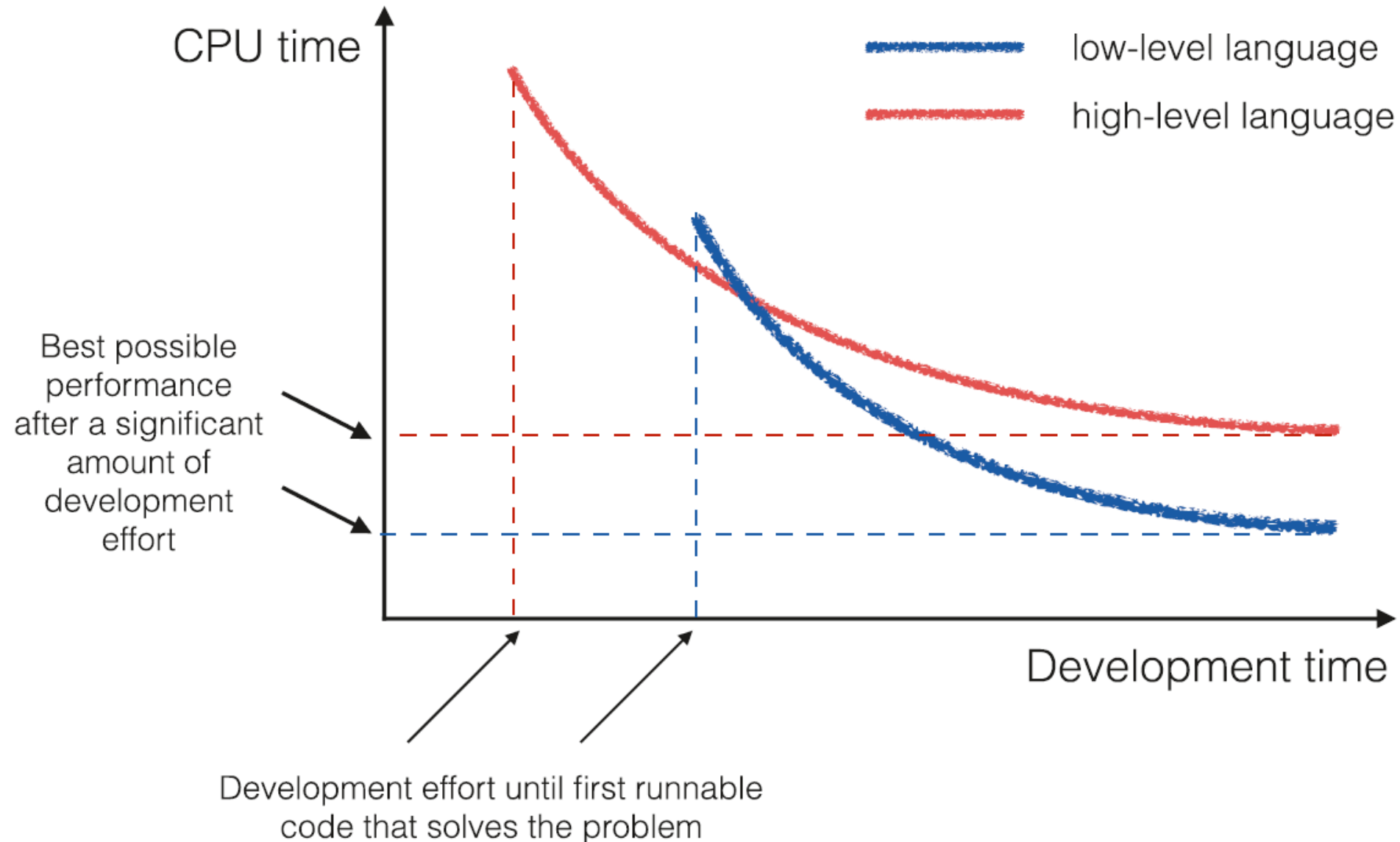
Converts the source code to machine code as you run the program (line by line)
Some programming languages (C/ C++/ Java) are compiled entirely prior to running

How do you think these affect speed?

How can you test that easily?



Trade-off between low- and high-level languages



Operating System (OS) independent:

Most interpreted languages are platform independent
(compiled languages: JDK and the .NET framework make Java and C# platform independent)
Compatible with Major Platforms and Systems

Object-oriented:

Design around data & objects rather than functions & logic

Open Source:

Community support

Package variety

Download source code, develop packages, libraries etc. and modify them

Makes it suitable for scientific research (reproducibility)

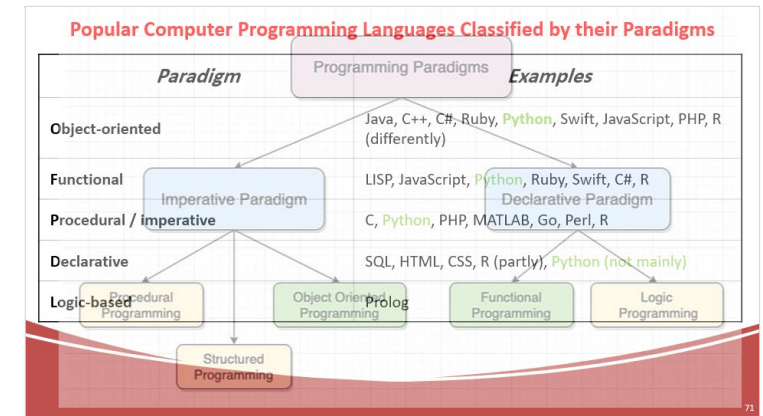
Zen and Style of Python

Easy to Learn

Concise

Focus on readability

PEP 8 & 20



Python Best Practices: PEP 8

Python has an official style guide called **PEP 8**.
PEP 8 stands for **Python Enhancement Proposal**.
PEP 8 gives recommendations for writing Python code that is readable, consistent, and easier to maintain.

Why this matters:

- You will read your code more often than you write it.
- Clear code is easier to debug.
- Consistent code is easier to share with others.
- Well structured code is easier to reuse in future projects.

Website: <https://www.python.org/pep/0008/>

Examples of PEP 8 Style

Some useful PEP 8 recommendations:

- Use meaningful variable names.
- Use lowercase words separated by underscores for variables and functions.
- Use lowercase words separated by underscores for constants.
- Use spaces around operators.
- When assigning default values to arguments don't use spaces.
- Use consistent indentation: 2 tabs, usually considered as 4 spaces, that can be set in your IDE.
- Keep lines reasonably short.
- Use comments where they help explain the logic.
- Separate code into clear sections.

Example:

```
# Example of PEP 8 style code
def main():
    """Main function"""
    # Do something here
    return 0

if __name__ == '__main__':
    main()
```

The Zen of Python: PEP 20

Python also has a set of broader programming principles called **PEP 20**, or **The Zen of Python**.

It includes principles such as:

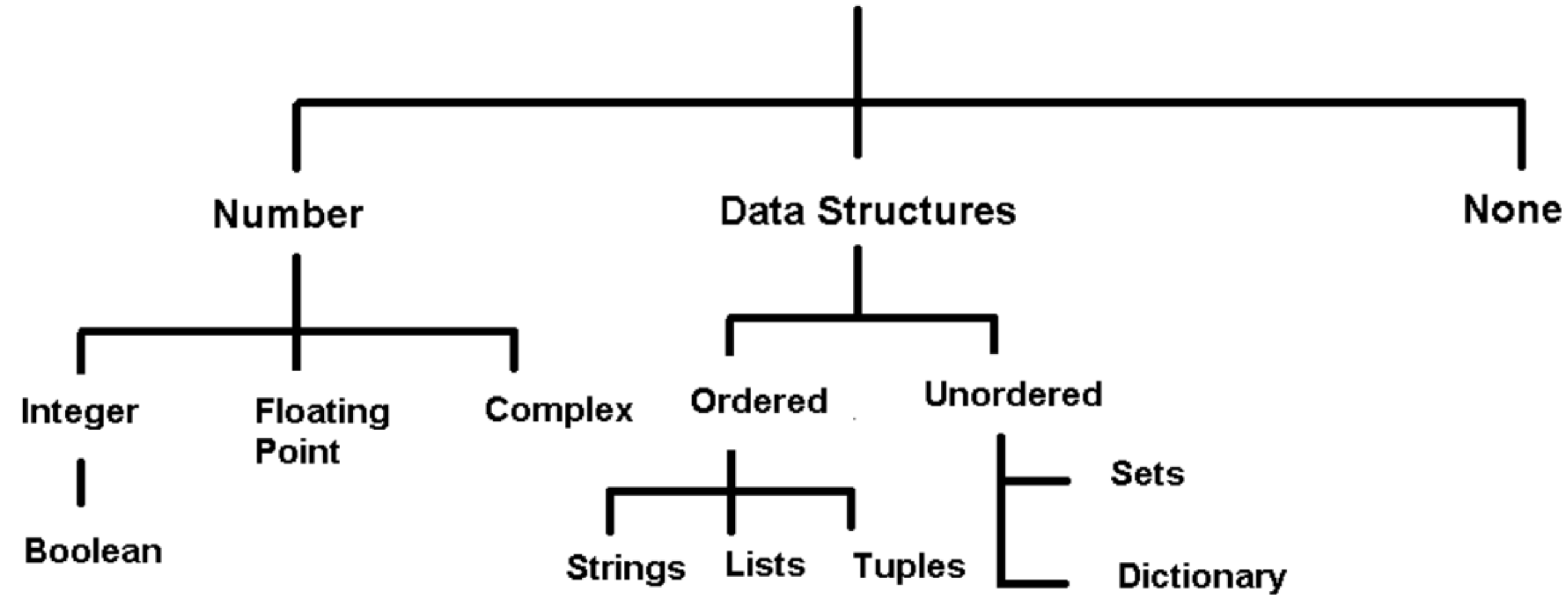
- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Readability counts.
- Errors should never pass silently.
- There should be one obvious way to do it.

You can display it in Python by running:

```
import sys
print(sys.path)
```

Website: <https://www.python.org/pep/0020/>

Python Data Types



Python Primitive Data Types

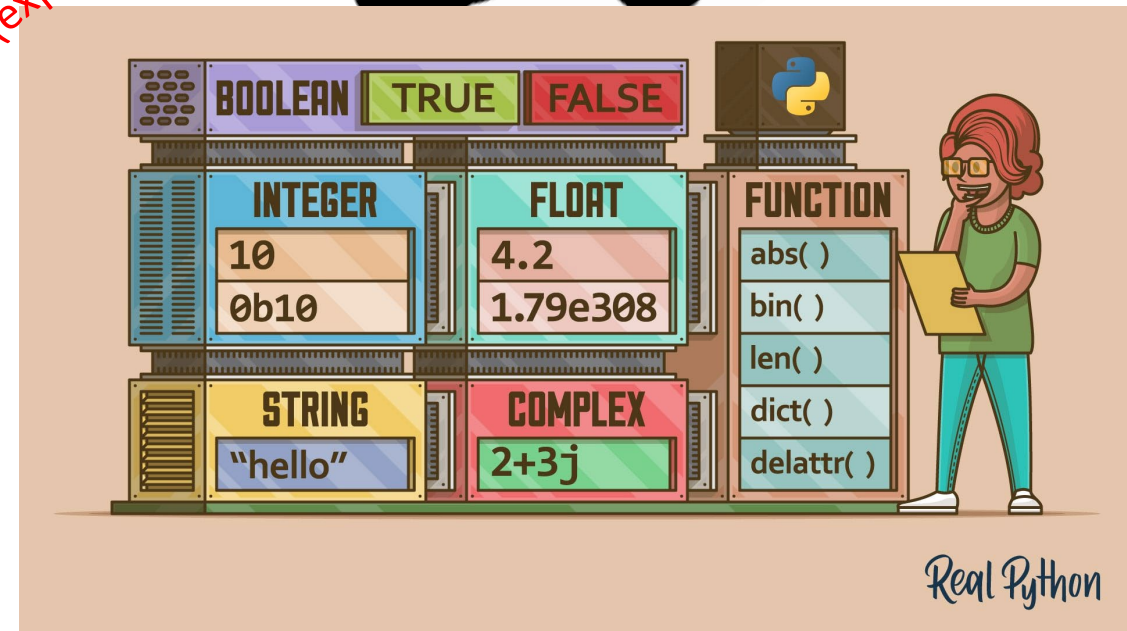
- **Strings:** Written characters/text: "Hello World", 'Q Step', 'three', "2"
- **Numbers**
 - **Integers:** 100, 2, 1892142012
 - **Floats:** 3.14, 2.9863
 - **Complex numbers:** $4 + 3j$, $-7j$ $i = \sqrt{-1}$
- **Boolean:** True, False

<https://docs.python.org/3/library/stdtypes.html>

Python is “Dynamically Typed”
You don’t need to deal with the
hassle of assigning data types
(we will see shortly in practice)

Strings are iterable and
immutable objects
(explained shortly)

...and i should care,
why?



The Data type Determines:

- The values that can be assigned to
- The operations that can be applied on

What is a Data Structure?

A **data structure** is a way of **organizing, storing, and managing data in a computer** so that it can be accessed and modified efficiently.

Why do we need data structures?

- Different problems require different ways of storing data.
- They help make programs:
 - Faster
 - Easier to write
 - Easier to maintain

Python Built-in Data Structures

List :

an ordered collection of one or more data items, not necessarily of the same type, *put in square brackets*.

[1,2,"C", True]

mutable

Can change/modify the values of a list

Tuple:

an ordered collection of one or more data items, not necessarily of the same type, *put in parentheses*.

(1,2,"C", True)

immutable

Cannot change/modify the values of a tuple

Strings are iterable and
immutable objects
(explained shortly)

Python Unordered Data Structures

Dictionary: an unordered collection of data in a key: value pair form

e.g. {'Dickens': 'Oliver Twist', 'Rowling': 'Harry Potter', 'Bronte': 'Jane Eyre', 'Seuss': 'Green Eggs and Ham', 'Shakespeare': 'Macbeth'}

Dictionary keys in Python do not need to be strings, or even from the same data type.

However, a dictionary key must be hashable.

In Python, an object is **hashable** if it has a hash value that never changes during its lifetime.

In practical terms, this almost always means the key must be an immutable data type.

- **Strings:** {"name": "Alice"}
- **Numbers (Integers, Floats):** {1: "First", 2.5: "Two and a half"}
- **Tuples:** {(40.7128, -74.0060): "New York City"} *(Note: The tuple can only contain other immutable types)*
- **Booleans:** {True: "Yes", False: "No"}

You cannot use mutable objects as keys because if their contents changed, their hash value would change, and Python would "lose" track of where it stored the data.

- **Lists:** {[1, 2]: "Numbers"} → **TypeError**
- **Dictionaries:** {'a': 1}: "Dict" → **TypeError**
- **Sets:** {[1, 2]: "Set"} → **TypeError**

Set: unordered collections of unique, mutable elements used to store non-duplicate items and perform efficient set operations such as union, intersection, and difference

Data Structure	Real-World Example	Why?
List	A shopping list <code>shopping_list = ["milk", "bread", "eggs"]</code>	Ordered collection; items can be added, removed, or changed.
Tuple	A date of birth (day, month, year) <code>date_of_birth = (15, 8, 1998)</code>	Fixed information that should not change.
Set	Students enrolled in a course <code>students = {"Alice", "Bob", "Charlie"}</code>	Each student appears only once; duplicates are not allowed.
Dictionary	A contact list (name → phone number) <code>contacts = { "Alice": "07123 456789", "Bob": "07987 654321" }</code>	Information is retrieved using a key rather than a position.

Data Structure	Common Methods	Purpose
String (str)	.lower(), .upper(), .title(), .strip(), .replace(), .split(), .join(), .find(), .startswith(), .endswith()	Text manipulation and searching
List (list)	.append(), .extend(), .insert(), .remove(), .pop(), .clear(), .index(), .count(), .sort(), .reverse(), .copy()	Adding, removing, searching, and reordering elements
Tuple (tuple)	.count(), .index()	Counting and locating elements (tuples are immutable)
Set (set)	.add(), .update(), .remove(), .discard(), .pop(), .clear(), .union(), .intersection(), .difference(), .symmetric_difference(), .issubset(), .issuperset()	Managing unique elements and set operations
Dictionary (dict)	.get(), .keys(), .values(), .items(), .update(), .pop(), .popitem(), .setdefault(), .clear(), .copy()	Accessing and modifying key-value pairs

Python Data Structure

```
graph TD; A[Python Data Structure] --> B[Built-in Python Data Structure]; A --> C[User-Defined Python Data Structure]; B --> D[List]; B --> E[Dictionary]; B --> F[Tuple]; B --> G[Set]; C --> H[Stack]; C --> I[Queue]; C --> J[Trees]; C --> K[LinkedLists]; C --> L[Graphs]; C --> M[HashMaps];
```

Built-in Python Data Structure

List

Dictionary

Tuple

Set

User-Defined Python Data Structure

Stack

Queue

Trees

LinkedLists

Graphs

HashMaps

There are a bunch of other data structures you can import from other modules/ packages (to be revisited later)

Python Variables

A variable is a name (reference) that points to an object in memory.

Think of a variable as a label attached to an object, rather than a box containing a value.

The assignment operator (=) is used to assign an object to a variable.

Use (==) for comparison

```
x = 5
```

```
name = "Alireza"
```

```
fruits = ["Apple", "Orange", "Pear"]
```

Python is case-sensitive!

Assignment creates a new reference, not necessarily a new object.

```
fruit_basket = fruits
```

```
fruit_basket.append("Banana")
```

```
print(fruits)
```



*What about
x and name?*

my_tuple = my_tuple + (4,) 34

Naming Variables

Names must start with a letter or underscore

Choose a sensible/ logical name:

✓ fruits = ["Apple", "Orange", "Pear"]

✗ Veg = ["Apple", "Orange", "Pear"]

✗ F = ["Apple", "Orange", "Pear"]

Separate words with an
underscore and use lower case:

✓ my_variable = 2

Do not use symbols:

! @ # \$ %

Do not start a variable name with a digit:

✗ 7_day = 20

✓ day_7 = 20

You cannot use keywords:

Keyword	Category / Use Case	Description	Keyword	Category / Use Case	Description	Keyword	Category / Use Case	Description
True	Boolean Value	Represents the boolean truth value.	while	Looping	Begins a while loop (runs as long as a condition is true).	raise	Exception Handling	Manually triggers/forces an exception to occur.
False	Boolean Value	Represents the boolean false value.	break	Loop Control	Terminates the loop prematurely.	assert	Debugging	Tests a condition; raises an AssertionError if the condition is false.
None	Special Constant	Represents the absence of a value or a null value.	continue	Loop Control	Skips the rest of the current loop iteration and moves to the next one.	import	Modules	Imports modules or packages into the current script.
and	Logical Operator	Returns True if both statements are true.	pass	Placeholder	A null statement; does nothing, used as a placeholder for future code.	from	Modules	Used with import to grab specific parts of a module.
or	Logical Operator	Returns True if at least one statement is true.	def	Functions	Used to define a function.	as	Modules / Context	Creates an alias (alternative name) for a module or context manager.
not	Logical Operator	Reverses the boolean result (not true becomes false).	return	Functions	Exits a function and optionally passes back a value.	with	Context Manager	Wraps the execution of a block of code with methods defined by a context manager.
is	Identity Operator	Tests if two variables point to the exact same object in memory.	lambda	Functions	Used to create small, anonymous (one-line) functions.	global	Variable Scope	Declares that a variable inside a function is global.
in	Membership Operator	Checks if a value is present in a sequence (list, string, tuple).	yield	Generators	Pauses a function and returns a generator value.	nonlocal	Variable Scope	Declares that a variable inside a nested function belongs to the outer function.
if	Conditional	Begins a conditional statement.	class	OOP	Used to define a new user-defined class.	del	Object Deletion	Deletes references to objects (variables, list items, etc.).
elif	Conditional	Short for "else if"; checks another condition if the previous ones were false.	try	Exception Handling	Starts a block of code to test for errors.	async	Asynchronous	Declares a function as an asynchronous coroutine.
else	Conditional / Loop	Executes if previous conditions aren't met (or if a loop finishes without a break).	except	Exception Handling	Catches and handles errors raised in the try block.	await	Asynchronous	Pauses the execution of an async function until a promise/future resolves.
for	Looping	Begins a for loop (used for iterating over a sequence).	finally	Exception Handling	Executes code after try/except, regardless of whether an error occurred.			

Special Naming Conventions

Convention	Example	Meaning
<code>_name</code>	<code>_internal</code>	Intended for internal use
<code>NAME</code>	<code>PI = 3.14159</code>	Constant (by convention)
<code>__name</code>	<code>__value</code>	Name mangling inside classes
<code>name_</code>	<code>class_</code>	Avoids keyword conflicts

```
class Student:
    def __init__(self):
        self.__grade = 85
```

Python automatically changes its internal name to reduce accidental access from outside the class.

```
s = Student()
print(s.__grade)    # AttributeError
But this works:
print(s._Student__grade)
```



Getting started with Python on your machine

- Installing Python
- Installing Anaconda (It installs Python separately)
- Creating environments
 - Conda env
 - venv
- Installing packages
- Running code
 - From the command prompt
- In IDEs
 - Spyder
 - VSCode
 - Sublime editor
 - Pycharm
- Online IDEs
 - Jupyter.org
 - Google colab

Home

Environments

Learning

Community

Anaconda Quick Start Environments
Jump into pre-configured environments by project or industry. Clean dependencies, faster development
Launch Your Environment

Documentation

Anaconda Blog



All applications

on

base (root)

Channels



PyCharm

The only Python IDE you need – built for data and AI/ML professionals. Supercharged with an AI-enhanced IDE experience. Free forever, plus one month of Pro included.

Install



Anaconda AI Navigator

Access various large language models (LLMs) curated by Anaconda, and start leveraging secure local AI today.

Install



Anaconda Toolbox

4.40.0

Anaconda Assistant

JupyterLab supercharged with a suite of Anaconda extensions, starting with the Anaconda Assistant AI chatbot.

Install



Anaconda Cloud Notebooks

Cloud-hosted notebook service from Anaconda. Launch a preconfigured environment with hundreds of packages and store project files with persistent cloud storage.

Launch



anaconda_powershell_prompt

1.1.0

Opens a PowerShell instance with conda activated (requires menuinst 2.1.1 or greater).

Launch



anaconda_prompt

1.1.0

Opens a terminal instance with conda activated (requires menuinst 2.1.1 or greater).

Launch



JupyterLab

4.2.5

An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.

Launch



Notebook

7.2.2

Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.

Launch



PyCharm Community

2024.1.2

An IDE by JetBrains for pure Python development. Supports code completion, listing, and debugging.

Launch



Qt Console

5.5.1

PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.

Launch



Spyder

5.5.1

Scientific PYTHON Development Environment. Powerful Python IDE with advanced editing, interactive testing, debugging and introspection features

Launch



VS Code

1.122.1

Streamlined code editor with support for development operations like debugging, task running and version control.

Launch



EduBlocks

Web-based coding platform from Anaconda designed for students. Learn Python coding through an interactive, block-based visual environment.



IBM watsonx

IBM watsonx is an enterprise-ready AI platform including a data store, model builder, and AI model management and monitoring.

ORACLE
Cloud Infrastructure

Oracle Data Science Service

OCI Data Science offers a machine learning platform to build, train, manage, and deploy your machine learning models on the cloud with your favorite open-source



PyScript

Code and share Python in the Browser. A vibrant community of makers, builders, and hackers building the next frontier of Python-powered web applications.



PythonAnywhere

Host, run, and code Python in the cloud! Get started for free.



CMD.exe Prompt

0.1.1

Run a cmd.exe terminal with your current environment from Navigator activated

Running Python from Anaconda Prompt

Open Anaconda Prompt

```
>python
```

You should see something similar to:

```
Python 3.14.0
```

This means the Python interpreter is running.

Your first Python program

Traditionally, programmers start with a "Hello, World!" program:

```
>print("Hello, World!")
```

Output:

```
Hello, World!
```

Common mistakes

- Missing quotation marks (" ")
- Missing parentheses (())
- Using mismatched quotes

Modern IDEs often help by automatically inserting matching brackets and quotes.

To leave the Python interpreter:

Option 1

```
>exit()
```

Option 2

```
>quit()
```

Option 3 (Windows)

Press:

```
Ctrl + Z
```

You will return to the command prompt.

ipython is the interactive computing engine that underpins Jupyter Notebook (that we will use later), providing features such as tab completion (intellisense), command history, and rich output.

```
>ipython
```

If you can't access it make sure it is installed using:

```
>pip install ipython
```

If you still have trouble, try:

```
>python -m IPython
```

Creating Your First Python Script

Instead of typing code interactively, we usually save it in a file.

Create a file

Open a text editor or IDE and enter:

```
print("Hello, World!")
```

Save the file

Create a folder:

```
Documents/  
└─ python_tutorial/
```

Save the file as:

```
say_hello.py
```

Running a Python Script

Navigate to the folder containing the file, for example:

```
>cd Documents\python_tutorial
```

Run the script:

```
>python say_hello.py
```

Output:

```
Hello, World!
```

Why use scripts?

- Code is saved permanently
- Easier to edit and share
- Suitable for larger programs

Use underscores instead of spaces in the file/ folder names, o.w.:

```
>python "say hello.py"
```

or

```
>python "C:\Users\Ali\Documents\say hello.py"
```

Useful Command Prompt Commands

Change drive:

```
>D:
```

Change to a folder:

```
>cd folder_name
```

Move up one level:

```
>cd ..
```

Or

```
>cd..
```

Go to the root directory:

```
>cd \
```

Or

```
>cd\
```

Display files and folders

```
>dir
```

Display files folders page-by-page

```
>dir/p
```

Run a Python file

```
>python documents\say_hello.py
```

Why Scripts Matter for Research and HPC

Beyond bothering you guys!

Scripts become especially valuable when projects grow larger or need to be run repeatedly.

Separation of Concerns

Keep the following separate:

- **Code** → what the computer should do
- **Parameters** → settings controlling the analysis
- **Data** → inputs to be analysed
- **Results** → outputs generated by the program

```
project/
├── analysis.py
├── config.json
├── data.csv
└── results/
```

Advantages

- ✓ Easier to reproduce analyses
- ✓ Easier to share with collaborators
- ✓ No need to edit code when parameters change
- ✓ Supports automation and batch processing
- ✓ Enables version control (Git)

Example

Instead of hard-coding:

```
n_iterations = 10000
```

```
learning_rate = 0.01
```

store them in:

config.json

```
{
    "n_iterations": 10000,
    "learning_rate": 0.01
}
```

Looks like a dict, ain't it?

High-Performance Computing (HPC)

Many research projects are too large to run on a personal computer.

Typical HPC workflow:

1. Develop and test locally.
2. Upload script(s) to the cluster.
3. Upload configuration and data files.
4. Submit a job.
5. Retrieve results when the job finishes.

Python Script + Config File + Data



HPC Cluster



Results

The same script can then be reused with different settings without modifying the source code.

Installing Packages & Using environments

Environments- what are they and why do we use them?

- Packages have **dependencies** to other packages that they need to work
- When installing multiple packages, one packages may require a **different versions of a dependent package to another**
- These difference dependencies need to be **resolved** so all packages work correctly
- When you have multiple projects, using lots of different packages resolving these is trickier.
- So, we create virtual environments, which group all the packages for a specific project together.



Python Environments and Package Management

A Python environment is an isolated workspace that contains:

- A specific Python version
- Installed libraries/packages
- Project-specific dependencies

Without environments:

- Package version conflicts can occur
- One project may break another
- Reproducibility becomes difficult

Example

Project A:

```
pandas==1.5
```

Project B:

```
pandas==2.3
```

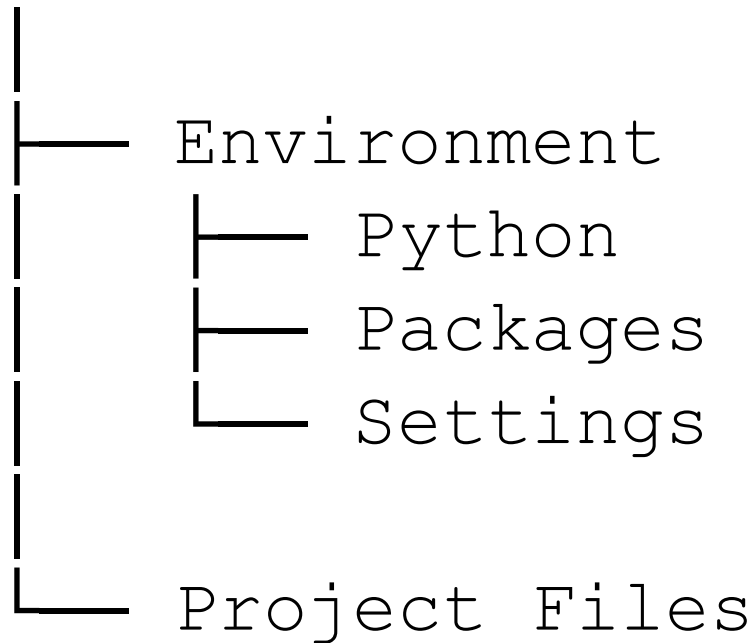
A virtual environment allows both projects to coexist on the same machine.

What Is a Virtual Environment?

Think of it as a separate Python installation

Each environment contains:

Project/



Benefits

- ✓ Isolates dependencies
- ✓ Improves reproducibility
- ✓ Prevents version conflicts
- ✓ Makes collaboration easier
- ✓ Simplifies deployment

Environment Management Tools

Common Options

Tool	Description
venv	Built into Python
Conda	Popular in data science
Poetry	Dependency management
Pipenv	Environment + package management

For Data Analysis Recommended: Conda

- Easy installation
- Handles non-Python dependencies
- Popular in scientific computing
- Works well with NumPy, Pandas, SciPy, PyTorch, etc.

Creating an Environment with venv

Create

```
>python -m venv myenv
```

Activate

Windows:

```
>myenv\Scripts\activate
```

Linux/macOS:

```
>source myenv/bin/activate
```

Install Packages

```
>pip install numpy pandas matplotlib
```

Deactivate

```
>deactivate
```

Creating an Environment with Conda

Create Environment:

```
>conda create --name myenv
```

```
>conda create -n myenv
```

```
>conda create -n myenv python=3.14 numpy=2.4
```

Activate:

```
>conda activate myenv
```

Prompt changes from:
(base)

Deactivate:

```
>conda deactivate
```

Prompt changes back to:
(base)

Managing Conda Environments

List Available Environments:

```
>conda env list
```

Remove an Environment:

```
>conda remove --name myenv --all
```

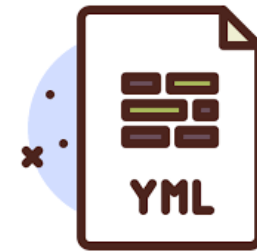
Export Environment:

```
>conda env export > environment.yml
```

Recreate Environment:

```
>conda env create -f environment.yml
```

This helps others reproduce your work exactly.



Installing Packages

```
>pip install package_name
```

```
>conda install package_name
```

Using pip

```
>pip install pandas==3.0
```

```
>pip install numpy pandas matplotlib
```

```
>pip install "numpy>=2.0,<3.0"
```

Using Conda

```
>conda install pandas
```

```
>conda install numpy pandas=3.0 matplotlib
```

```
>conda install "numpy>=2.0,<3.0"
```

Inside a Jupyter Notebook

```
>!pip install numpy
```

The exclamation mark (!) tells Jupyter to execute a shell command.

Practical Recommendation

- Use **Conda** to create environments.
- Use **Conda** packages when available.
- Use **pip** only when a package is not available through Conda.

pip

Python packages only

Comes with Python

Very large package repository

Standard Python tool

Conda

Python + system libraries

Comes with Anaconda/ Miniconda

Excellent scientific ecosystem

Popular in data science

Beyond Basic Installation

Install into a specific conda environment

```
>conda install -n myenv scipy
```

Install directly from GitHub

```
>pip install git+https://github.com/user/project.git
```

Upgrade a Package

```
>pip install --upgrade pandas
```

Force reinstall

```
>pip install --force-reinstall matplotlib
```

Install a package for development

```
>pip install -e .
```

Install from a particular conda channel

```
>conda install -c conda-forge xarray
```

Reproducible Research

Bad Practice

"It works on my computer."

Good Practice

Share:

`environment.yml`

or

`requirements.txt`

Example

```
>pip freeze > requirements.txt
```

Reinstall later:

```
>pip install -r requirements.txt
```

Installing a Project from Your Own Directory in Editable Mode

When developing a Python project, your code changes frequently.

Standard Installation

```
>pip install .
```

the current directory (folder)

- Copies the package into the Python environment.
- Changes to the source code are **not reflected automatically**.
- You must reinstall after every modification:

```
>pip install .
```

-e = --editable

Editable Installation

```
>pip install -e .
```

- Creates a link to your source code instead of copying it.
- Changes to the source files are **available immediately**.
- No reinstallation is required.

Example Workflow

Modify code

↓

Save file

↓

Run program again

↓

Changes take effect immediately

Why Is This Useful?

- Faster development and testing
- Common practice for research and software projects
- Frequently used after cloning a GitHub repository

```
>git clone <repository>
```

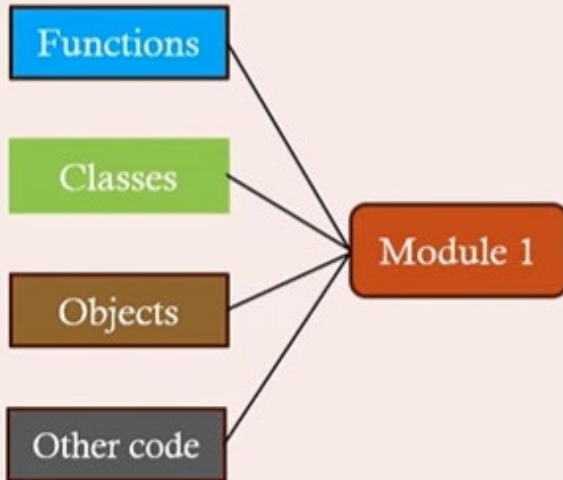
```
>cd <repository>
```

```
>pip install -e .
```

Python Module vs Package vs Library vs Framework

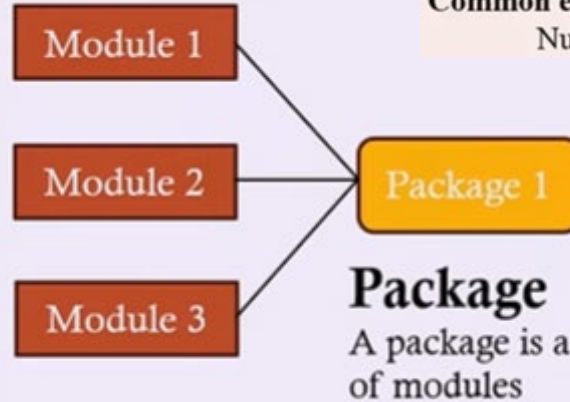
Module

A file saved with a .py extension, that consists of functions, classes & variables that can be imported into other scripts

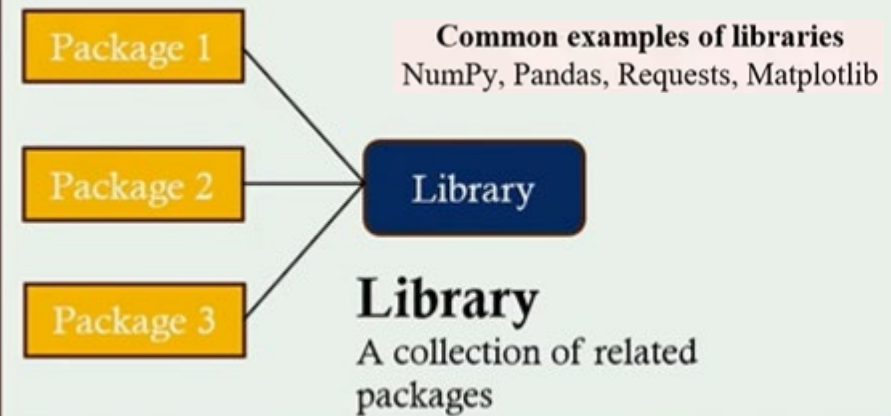


Common examples of modules
math, datetime, random

Common examples of packages
NumPy, Pandas

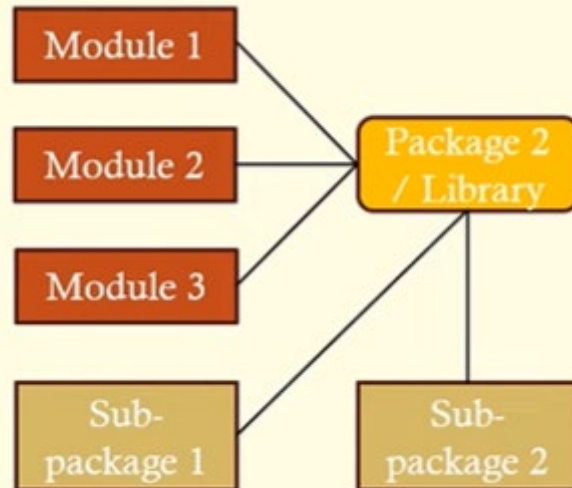


Common examples of libraries
NumPy, Pandas, Requests, Matplotlib

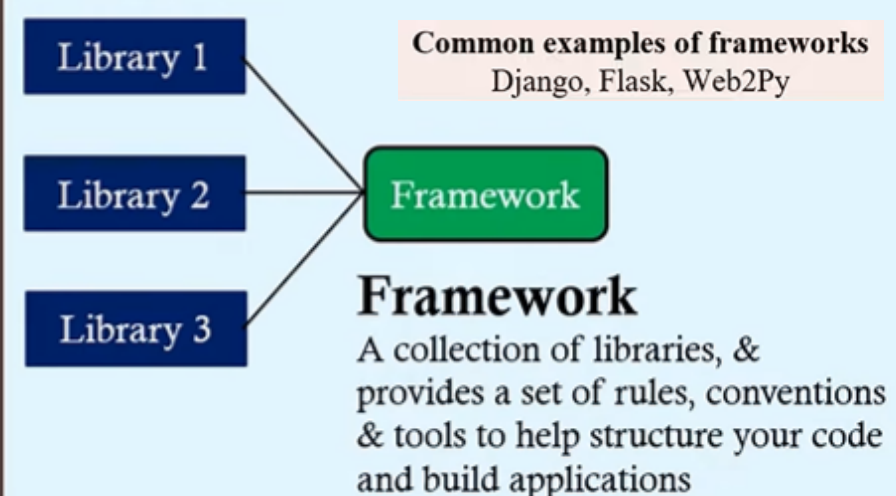


Package / Library

A package can be re-classified as a library because it may contain sub-packages & modules



Common examples of frameworks
Django, Flask, Web2Py



Installing and Running the Jupyter Notebook/ Lab

First navigate to your desired directory (folder):

```
>cd directory_address
```

```
>jupyter lab --notebook-dir="D:\directory_address"
```

Activate your environment:

```
>conda activate myenv
```

Try launching Jupyter:

```
>jupyter notebook
```

If Jupyter is not installed, install it.

Using Conda

```
>conda install jupyter
```

Using Pip

```
>pip install jupyter
```

Then for opening the Jupyter notebook run:

```
>jupyter notebook
```

And for opening the Jupyter lab run:

```
>jupyter lab
```

You can always switch between the notebook and the lab later

You can also access the Jupyter notebook/ lab through the Anaconda Navigator.

You can select your desired environment (kernel) when you create a new file.

However, navigation to your desired directory should be done through some code:

```
import os
```

```
os.chdir(r"D:\Projects")  
print(os.getcwd())
```

<https://github.com/alireza-hosseinzadeh/intro-python-practicals>

Errors

Errors Are Normal!

- **Every programmer encounters errors**
- Errors are not a sign of failure; they are part of software development.
- The challenge is not avoiding errors but learning how to find and fix them efficiently.

Error Type	When Found	Example
Compile-Time Error	Before execution	Missing colon after <code>if</code>
Runtime Error	During execution	Division by zero
Logical Error	Program runs but gives wrong result	Incorrect formula

Compile-Time (Syntax) Errors

- Violations of Python's grammar rules.
- Python cannot even start running the program.

- **Example**

```
if age > 18  
    print("Adult")
```

- Output:

```
SyntaxError: expected ':'
```

Common Causes

- Missing `:`
- Missing parentheses
- Incorrect indentation
- Misspelled keywords

How to Fix

- Read the error message carefully
- Check the line number
- Use an IDE (VS Code, PyCharm)
- Use LLMs

Runtime Errors

The program starts successfully but crashes while running.

Example 1:

```
x = 10  
y = 0
```

```
print(x / y)
```

Output:

```
ZeroDivisionError
```

Examples 2:

```
numbers = [1, 2, 3]  
print(numbers[5])
```

Output:

```
IndexError
```

Examples 3:

```
print(Numbers)
```

Output:

```
NameError
```

Exception Handling

Python allows us to recover gracefully from some errors.

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("Cannot divide by zero")
```

Output:

```
Cannot divide by zero
```

Benefits

- Prevent crashes
- Provide useful feedback
- Improve robustness

Logical Errors: The Most Dangerous Errors

The program runs without crashing.

The result is simply wrong.

Example

Suppose we want the average:

```
marks = [60, 70, 80]
```

```
average = sum(marks) / 2
```

Output:

```
105
```

No error occurs, but the answer is incorrect.

Why Are They Difficult?

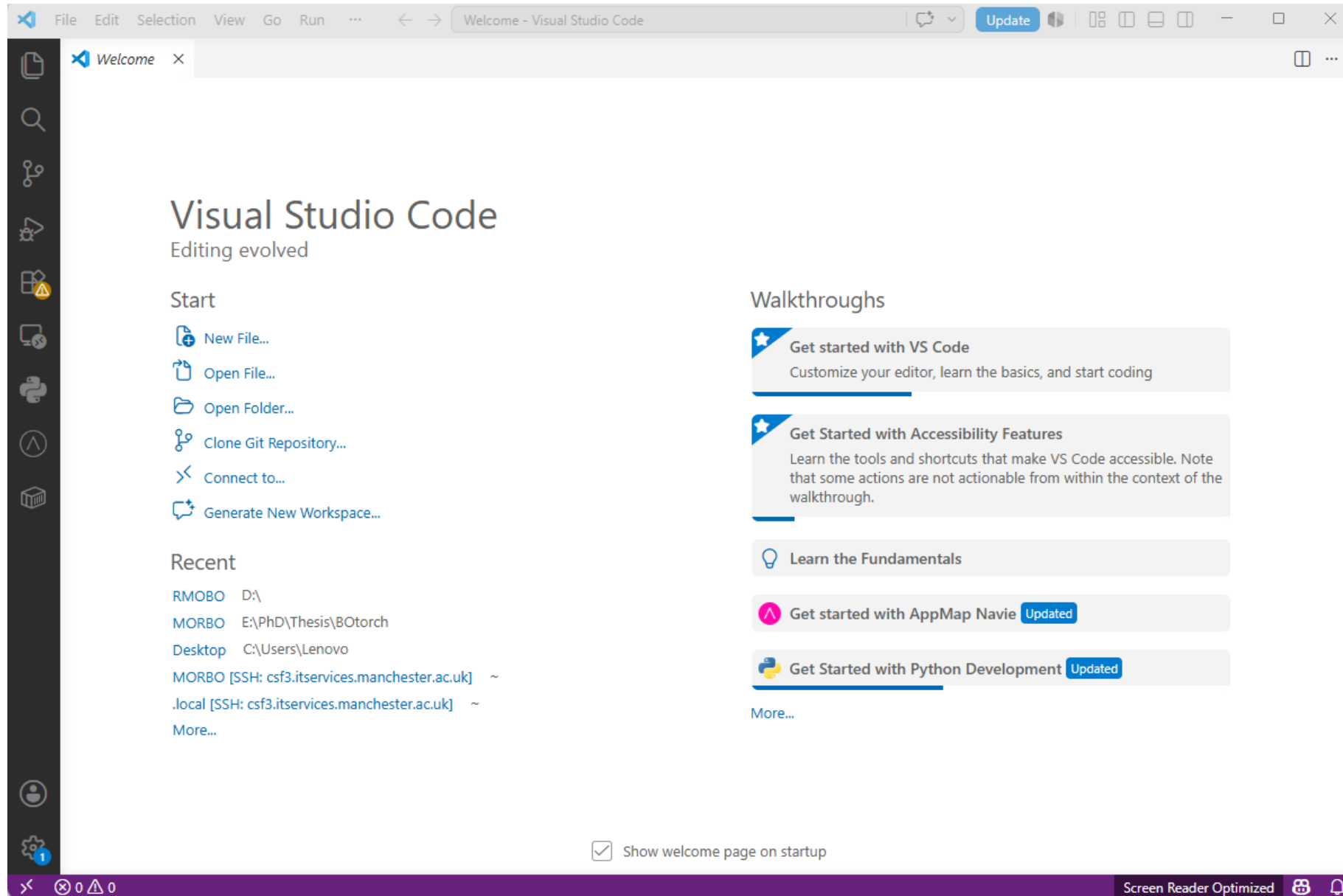
- No warning message
- No crash
- Often discovered much later

- Interactive Debugging
- Debugging with Breakpoints
- Logging
- Writing tests
- Rubber Ducking
- Using LLMs

Runtime and Compile-time Error Examples

Error Type	When Detected?	Example
SyntaxError	Before execution	Missing :
IndentationError	Before execution	Incorrect indentation
NameError	During execution	Undefined variable
TypeError	During execution	Invalid operation between types
IndexError	During execution	Invalid list index
KeyError	During execution	Missing dictionary key
FileNotFoundError	During execution	Missing file

Visual Studio Code (VS Code)



Using VS Code with Conda Environments

Open VS Code From the Anaconda

Prompt:

Open current folder:

```
>code .
```

Open a specific file:

```
>code filename.py
```

Open VS Code without a folder:

```
>code
```

- You can also launch VS Code from the Anaconda Navigator.
- Alternatively, you can install VS Code independently and use it without Anaconda.

Selecting the Python Environment

Press:

```
Ctrl (Cmd) + Shift + P
```

Search for:

```
Python: Select Interpreter
```

Choose your Conda environment.

You can also select your desired env from the pull-down menu at the top right corner.

<https://github.com/alireza-hosseinzadeh/intro-python-practicals>

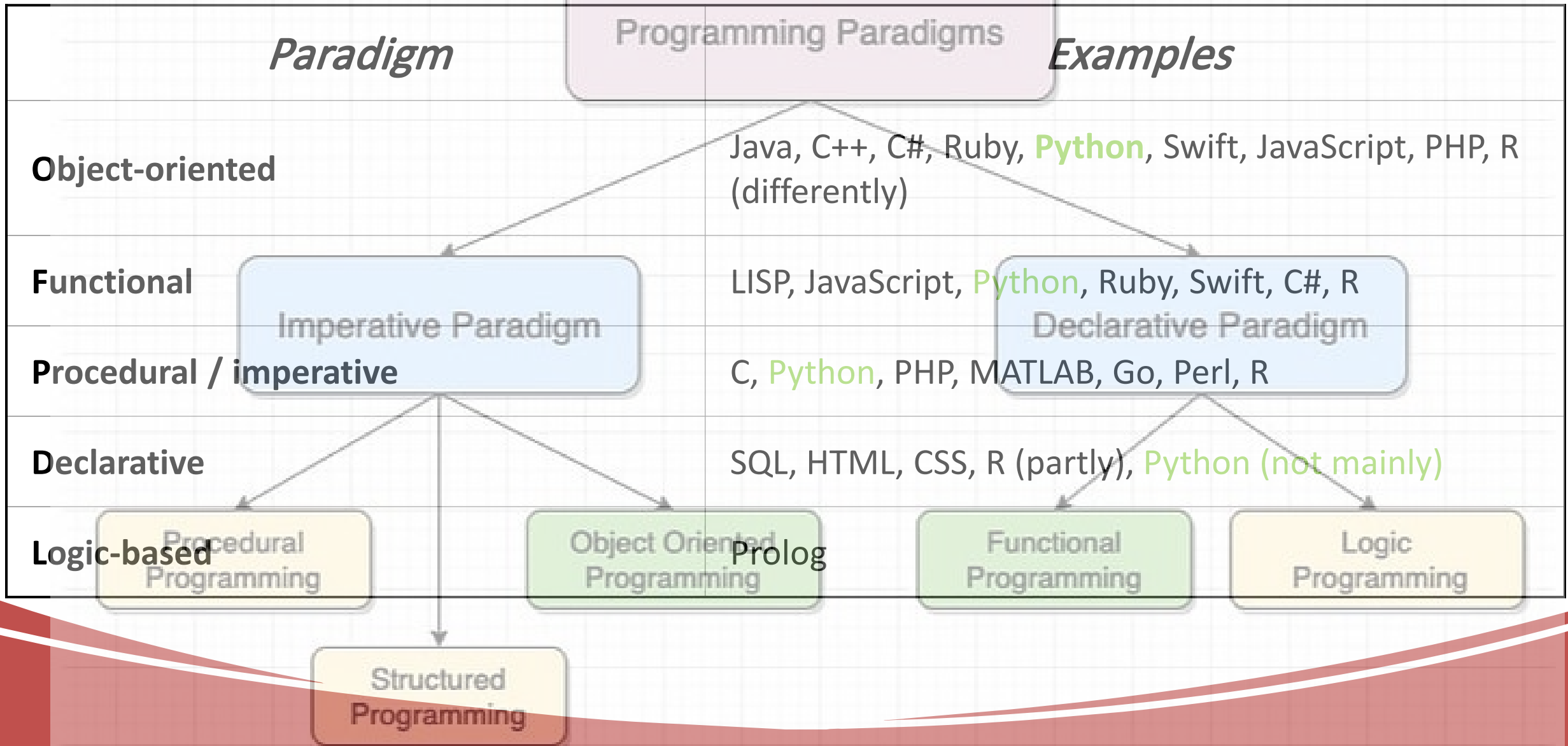
Tasks to pursue later

- Explore and sign-up for github: <https://github.com/>
- Consider how these elements may help you on your journey.
- Work through a range of tutorial information:
 - ✓ <https://ourcodingclub.github.io/tutorials/git/>
 - ✓ <https://docs.github.com/en/get-started/quickstart/hello-world>
 - ✓ <https://rss.org.uk/RSS/media/News-and-publications/Publications/Reports%20and%20guides/A-Guide-for-Ethical-Data-Science-Final-Oct-2019.pdf>

<https://github.com/search?q=introduction+python&type=repositories&p=2>

Appendix

Popular Computer Programming Languages Classified by their Paradigms



Keyword	Category / Use Case	Description	Keyword	Category / Use Case	Description	Keyword	Category / Use Case	Description
True	Boolean Value	Represents the boolean truth value.	while	Looping	Begins a while loop (runs as long as a condition is true).	raise	Exception Handling	Manually triggers/forces an exception to occur.
False	Boolean Value	Represents the boolean false value.	break	Loop Control	Terminates the loop prematurely.	assert	Debugging	Tests a condition; raises an AssertionError if the condition is false.
None	Special Constant	Represents the absence of a value or a null value.	continue	Loop Control	Skips the rest of the current loop iteration and moves to the next one.	import	Modules	Imports modules or packages into the current script.
and	Logical Operator	Returns True if both statements are true.	pass	Placeholder	A null statement; does nothing, used as a placeholder for future code.	from	Modules	Used with import to grab specific parts of a module.
or	Logical Operator	Returns True if at least one statement is true.	def	Functions	Used to define a function.	as	Modules / Context	Creates an alias (alternative name) for a module or context manager.
not	Logical Operator	Reverses the boolean result (not true becomes false).	return	Functions	Exits a function and optionally passes back a value.	with	Context Manager	Wraps the execution of a block of code with methods defined by a context manager.
is	Identity Operator	Tests if two variables point to the exact same object in memory.	lambda	Functions	Used to create small, anonymous (one-line) functions.	global	Variable Scope	Declares that a variable inside a function is global.
in	Membership Operator	Checks if a value is present in a sequence (list, string, tuple).	yield	Generators	Pauses a function and returns a generator value.	nonlocal	Variable Scope	Declares that a variable inside a nested function belongs to the outer function.
if	Conditional	Begins a conditional statement.	class	OOP	Used to define a new user-defined class.	del	Object Deletion	Deletes references to objects (variables, list items, etc.).
elif	Conditional	Short for "else if"; checks another condition if the previous ones were false.	try	Exception Handling	Starts a block of code to test for errors.	async	Asynchronous	Declares a function as an asynchronous coroutine.
else	Conditional / Loop	Executes if previous conditions aren't met (or if a loop finishes without a break).	except	Exception Handling	Catches and handles errors raised in the try block.	await	Asynchronous	Pauses the execution of an async function until a promise/future resolves.
for	Looping	Begins a for loop (used for iterating over a sequence).	finally	Exception Handling	Executes code after try/except, regardless of whether an error occurred.			

Useful Command Prompt Commands

Change drive:

```
>D:
```

Change to a folder:

```
>cd folder_name
```

Move up one level:

```
>cd ..
```

Or

```
>cd..
```

Go to the root directory:

```
>cd \
```

Or

```
>cd\
```

Display files and folders

```
>dir
```

Display files folders page-by-page

```
>dir/p
```

Run a Python file

```
>python documents\say_hello.py
```

Python Best Practices: PEP 8

Python has an official style guide called **PEP 8**.

PEP stands for **Python Enhancement Proposal**.

PEP 8 gives recommendations for writing Python code that is readable, consistent, and easier to maintain.

Why this matters:

- You will read your code more often than you write it.
- Clear code is easier to debug.
- Consistent code is easier to share with others.
- Well-structured code is easier to reuse in future projects.

Website: <https://peps.python.org/pep-0008/>

75

Examples of PEP 8 Style

Some useful PEP 8 recommendations:

- Use meaningful variable names.

`student_age` instead of `x`

Use lowercase words with underscores for variables and functions.

`clean_data`

`calculate_mean`

- Use spaces around operators.

`total = price * quantity`

- When assigning default values to arguments don't use spaces.

`arg1=3`

- Use consistent indentation: 1 tab, usually considered as 4 spaces, that can be set in your IDE.

- Keep lines reasonably short.

- Use comments where they help explain the logic.

- Separate code into clear sections.

Example:

```
# Calculate average income after removing missing values
cleaned_data = data.dropna(subset=["income"])
average_income = cleaned_data["income"].mean()
```

76

The Zen of Python: PEP 20

Python also has a set of broader programming principles called **PEP 20**, or **The Zen of Python**.

It includes principles such as:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Readability counts.
- Errors should never pass silently.
- There should be one obvious way to do it.

You can display it in Python by running:

```
import this
```

Website:

<https://peps.python.org/pep-0020/>

77

Python Best Practices: PEP 8

Python has an official style guide called **PEP 8**.

PEP stands for **Python Enhancement Proposal**.

PEP 8 gives recommendations for writing Python code that is readable, consistent, and easier to maintain.

Why this matters:

- You will read your code more often than you write it.
- Clear code is easier to debug.
- Consistent code is easier to share with others.
- Well-structured code is easier to reuse in future projects.

Website: <https://peps.python.org/pep-0008/>

Examples of PEP 8 Style

Some useful PEP 8 recommendations:

- Use meaningful variable names.

`student_age` instead of `x`

Use lowercase words with underscores for variables and functions.

`clean_data`

`calculate_mean`

- Use spaces around operators.

`total = price * quantity`

- When assigning default values to arguments don't use spaces.

`arg1=3`

- Use consistent indentation: 1 tab, usually considered as 4 spaces, that can be set in your IDE.
- Keep lines reasonably short.
- Use comments where they help explain the logic.
- Separate code into clear sections.

Example:

```
# Calculate average income after removing missing values
cleaned_data = data.dropna(subset=["income"])
average_income = cleaned_data["income"].mean()
```

The Zen of Python: PEP 20

Python also has a set of broader programming principles called **PEP 20**, or **The Zen of Python**.

It includes principles such as:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Readability counts.
- Errors should never pass silently.
- There should be one obvious way to do it.

You can display it in Python by running:

```
import this
```

Website:

<https://peps.python.org/pep-0020/>